

实验报告 / Experiment Report — FAB Level-B

Evolution Loop: evolve agent 对比、harness 修复与难度阶梯

日期/Date: 2026-07-07 · 实验/Experiment: Phase-5 FAB Level-B evolution (2026-07-02 → 07-07) · 分支: worktree-phase5-levelb-mechanism

0. 实验设定 / Setup (先读这节，正文所有术语在此定义)

要研究的问题： 让一个 LLM agent (**evolve agent**) 像工程师一样去"修"另一个 LLM agent (**worker**) ——通过编辑 worker 的配置文件、提示词和工具代码来提升它的任务表现。这套机制叫 **Level-B evolution loop** (复现自 AHE, *Agentic Harness Engineering* 论文的核心循环)。

实验设计： 我们先人为**削弱**一个能正常工作的 worker (拆掉它的部分工具)，得到一个低分的起点 (**seed**)，然后看 evolve agent 能不能通过诊断失败原因、编辑 worker 文件，把被拆掉的能力**恢复**回来。"能恢复多少分"就是这套机制的效果度量 (**headroom recovery**)。

Worker 与 benchmark： worker 是一个做 **FAB benchmark** (Finance Agent Benchmark, 27 道 SEC-filing 金融研究题，如"某公司 10-K 里 loyalty 会员占比是多少") 的 agent，答案由 LLM judge 按每题的 rubric (评分细则) 打 0-1 分，整个 run 的分数 = 27 题平均分。完整版 worker 有 6 个工具；**削弱版 (seed)** 只留 2 个通用工具 (fetch_page 抓网页、web_search 搜索)，**拆掉 4 个 SEC 专用工具：**

被拆掉的工具	作用
edgar_search	在 SEC EDGAR (美国证监会文件库) 全文搜索 filing
company_filings	列出某公司的 filing 清单和 URL
retrieve_from_filing	在一份超长 filing 里按关键词取出相关段落 (fetch_page 只能抓前 4 万字符，10-K 动辄几十万字，没有它就读不到文件深处——这是 4 个里最关键的)
price_history	取股价历史

拆法 (难度阶梯)： T0/mid-tier = 工具的实现代码还留在 worker 目录的 tools/ 里，只是没在配置 (agent.yaml) 里挂载，且有注释点名它们 (重新挂上即可恢复)；T1 = 同上但删掉点名注释 (须自己探索发现)；T2 = 连实现代码也删掉 (须从零写代码)。

循环流程 (每一轮 iteration)：

```
seed eval (跑 27 题拿基线分)
  → diagnose (debugger 读失败题，产出根因诊断，如"缺深读 filing 的能力")
  → evolve agent 依诊断编辑 worker 副本 (candidate)
```

- candidate eval (编辑后的 worker 重跑 27 题)
- keep / rollback: candidate 平均分超过现任版本 0.05 (noise floor) 才保留, 否则回滚

固定不变的部分: worker 的答题模型固定为 deepseek-v4-pro, judge 固定为 qwen3.7-plus (每题打 2 遍取中位数, 压评分随机性); 全部 worker 在 E2B 云 VM 里运行。**唯一实验变量 = evolve agent 用什么模型 (deepseek-v4-pro vs GLM-5.2) + 循环本身的设计。**

0.1 术语表 / Terminology (正文按此使用)

术语	含义
kept / rollback	一轮编辑被保留 / 被回滚。"kept 2/10" = 10 轮里有 2 轮的编辑被保留
noise floor (0.05)	keep 门槛: 因为 worker 生成和 judge 打分都有随机性, 同一 worker 重测分数会漂 ± 0.03 左右, 所以要求提升超过 0.05 才认为是真改进
infra-0 (基础设施 0 分)	某题得 0 分不是因为答错, 而是因为没跑成——云端限流 (HTTP 429)、sandbox 断连、网络超时等瞬时故障把该题记成 0。它会假装成"编辑导致退步"
transient-masking / infra 掩盖	一次真实的改进 (如 +0.16) 被几个 infra-0 拉低总平均分, 导致被误判 rollback。"infra 运气" = 两次 run 的结果差异其实来自谁恰好被 infra-0 砸中, 而非能力差异
fair-subset keep	对 masking 的修复: keep 判决只在"seed 和 candidate 双方都正常跑完 (无 infra 故障)"的题目子集上比较平均分, 使任何 infra-0 无法再冒充退步
firewall (诊断防火墙)	旧设计: 为防评分答案泄漏, 禁止诊断提及任何具体数值 ("answer-free")。后证明它摧毁了关键诊断信号, 已移除 (防作弊改由编辑阶段的 LeakageGuard 承担)
信标 / signpost	seed 配置文件里一条点名"哪 4 个工具被拆掉了"的注释——相当于给 evolve agent 的提示。T1 实验专门删掉它来测真发现能力
探索行为	evolve agent 每轮 选择改什么 的模式: 「多样」= 不同轮尝试不同方向 (改提示词/加计算器/接检索工具); 「固着」= 每轮反复改同一个东西
calc.py / calculator	evolve agent 自己给 worker 写的计算器小工具 (算数用)。它治不了"读不到原文"的病, 所以在检索缺口面前是无效方向
verdict (MIXED/EFFECTIVE/HARMFUL)	对一轮编辑的判定: EFFECTIVE = 预测会修好的题真的修好了; MIXED = 有好有坏; HARMFUL = 预测全落空且出现退步
NOFW	"no-firewall" run 的代号: 使用移除防火墙 + 不限每轮编辑数的新循环跑的实验

术语	含义
format gate (GDPval 用)	GDPval 的题要求交付真实文件 (.xlsx/.pdf): 没产出要求类型的文件, 该题直接记 0 分

1. 实验过程与结果 / Process & Results

1.1 目标 / Goal

1. 回答 "deepseek-v4-pro vs GLM-5.2 谁是更好的 evolve agent";
2. 过程中修复 harness 的系统性缺陷 (infra 噪声、诊断 firewall、edit 预算);
3. 建立难度阶梯 (T0 重接 → T1 发现 → T2 代码合成) 并给 evolve agent 补上 `run_code` 自测能力;
4. 把 evolve agent 也搬上 E2B (双 agent 全云化, 为 GDPval 移植做准备)。

1.2 实验时间线与配置 / Timeline & Config

统一命令骨架 (各阶段只换 seed / iters / evolve model):

```
run.py --levelb --benchmark fab --seed-worker qea/worker_fab_weak_midtier[<tier>]
--execution e2b_full --concurrency 6 --n-tasks 27 --iters <N> --k 2
--evidence-mode ahe_corpus --noise-margin 0.05
```

阶段	日期	内容	关键 commit
A	07-02/03	首批 4 次 run (旧 firewalled loop) + infra 修复	f7b4e62 453846e 130afce f5d28bd
B	07-05	fair-subset keep 修复 + matched 复跑	17e8169
C	07-06	10-iteration 对比	—
D	07-06	根因定位 (debugger 误诊) → un-firewall + 放开 edit	f245eb6
E	07-06	T1 (无信标) 发现测试	6e0a6f2
F	07-06/07	<code>run_code</code> 自测 + T2 代码合成测试	9e81249 01657bd
G	07-07	evolve agent 上 E2B (双 agent 全云化)	542a0ab

1.3 主数据表 / Headline Data

Seed baselines (各难度档的起点分, 27 题平均, 全部题目正常跑完、无 infra-0 污染): T0/mid-tier = **0.4748**; T1 (无信标) = **0.5337**; T2 (删实现) = **0.4289**。作为参照: 未削弱的完整 worker

≈ 0.618——即 mid-tier 留给恢复的空间约 0.14—0.20。

(A) 旧循环（带诊断防火墙 + 每轮只许改一处）

run	evolve agent	iters	kept	final	备注
4 次 run 合 计	deepseek	3—5	0/16	0.4748 (原地不 动)	其实找到过 +0.056/+0.059 的真改进，但每次都恰好有几题被 infra=0（限流/断连记 0）拉低平均分 → 被误判为退步而回滚
1 次	GLM-5.2	5	1/5	0.535	当时唯一被保留的编辑——后证明只是这次 run 恰好没被 infra=0 砸中（infra 运气），不是能力差异

结论：表面结果是"GLM 能恢复、deepseek 不能"，但复查发现 deepseek 的好编辑全被 infra=0 拉低均分误杀。→ 下一步动机：结果不可信，必须先让 keep 判决对 infra 故障免疫（fair-subset keep），再在干净条件下重比。

(B) fair-subset keep 修复后（infra=0 不再能冒充退步，见术语表）

run	evolve agent	kept	final
fair-keep	deepseek	1/4	0.541
fair-keep r1	GLM-5.2	1/4	0.546

结论：排除 infra 干扰后两个模型打平（~0.54）——(A) 里"GLM 赢"纯属 infra 运气（重算双方被掩盖的最佳编辑：deepseek +0.166 vs GLM +0.206，量级相当）。且两者都停在"保留 1 个编辑就上不去"的平台期。→ 下一步动机：5 轮可能太短、不足以分出高下或突破平台期——把轮数加到 10。

(C) 10-iteration 长跑（仍是旧的带防火墙循环）

run	kept	分数轨迹	探索行为（每轮改了什 么）
deepseek	2/10	0.475 → 0.538（第 6 轮加计算器被保留）→ 0.660（第 7 轮重新挂载 retrieve_from_filing ——终于接回了那个最关键的深读 filing 工具）	多样：先后尝试改提示词 / 加计算器 / 接检索工具 / 接股价工具 4 种方向
GLM	0/10	0.475 全程原地（最佳编辑仅 +0.014）	固着：10 轮全部在反复改同一个计算器文件，从未尝试接检索工具

结论：轮数拉长后 deepseek 靠探索多样性胜出（第 7 轮才试到检索工具、一举 +0.12）；GLM 固着在计算器上颗粒无收。→ 下一步动机：一个刺眼的疑问——最关键的检索工具，为什么 deepseek 要 7 轮才碰、GLM 永远不碰，两者却都在计算器上空转多轮？追查诊断内容后找到根

因：诊断防火墙把"数值接近但错"（检索问题的特征）抽象成了"计算不精确"，一直把 evolve agent 引向计算器。于是移除防火墙。

(D) 移除诊断防火墙 + 不限每轮编辑数（决定性修复，commit f245eb6；这类 run 代号 NOFW）

背景：查明 (A)(C) 收敛慢的根因是**诊断误导**——防火墙禁止诊断提数值，"答 72% 但正确是 75%"只能被抽象成"数值计算不精确"，于是诊断连续 5 轮说"缺计算器"，evolve agent 就一直造计算器（详见案例 1）。移除防火墙 + 允许一轮改多处后：

run	evolve agent	第 1 轮分数	第 1 轮动作
NOFW	deepseek	0.701（起点 0.475）	一轮同时接回全部 4 个 SEC 工具
NOFW	GLM-5.2	0.698	同样一轮接全 4 个
（后续轮次峰值）	deepseek	~0.728（第 3 轮）	之后进入平台期 ~0.70—0.73

结论：新循环第 1 轮就超过旧循环 7 轮的水平（0.70 vs 0.66），GLM 从"10 轮 0 保留"被完全解锁——此前的瓶颈在循环设计（诊断误导 × 一轮只许改一处），不在模型。→ **下一步动机**：但 seed 配置里有一条点名 4 个工具的信标注释，"一轮接全 4 个"可能只是照抄提示——删掉注释验证是不是真发现。

(E) T1 无信标实验（删掉点名工具的注释，检验是真发现还是照抄提示）

动机：(D) 里 evolve agent 一轮接全 4 个工具，可能只是照抄了配置文件里点名工具的信标注释。T1 把注释删掉再跑：

指标	NOFW（有信标）	T1（无信标）
第 1 轮接上的工具数	4/4	3/4（漏了 edgar_search；靠自己 ls tools/ 列目录 + 通读工具源码发现未挂载的函数）
同一 20 题子集上的分数	0.821	0.758

结论：发现能力是真的（无任何提示仍自主找到并接回 3/4 工具），信标的价值仅 $\approx +0.06$ 。※曾误报"T1 反超有信标版"——那是拿 T1 的部分数据（20 题偏易子集）对比全量 27 题的假象，同子集重算后已更正。→ **下一步动机**：T1 证明它能"找到并接回现成代码"，难度阶梯的下一档是"代码根本不存在"——它能不能从零把工具写出来？写代码没有运行反馈等于盲写，所以先给 evolve agent 补上 run_code（写完能跑一下自测）再上 T2。

(F) T2 代码合成实验（连工具实现代码也删掉，全 seed 无任何"这些工具存在过"的痕迹；evolve agent 新增 run_code 工具可运行代码自测；起点 0.4289）

维度	deepseek	GLM-5.2
是否从零写出检索工具	✅ 写出 sec_fact（+160 行，调 SEC 官方 company-facts 数字 API）	❌ 工具源码文件 5 轮从未改动

维度	deepseek	GLM-5.2
用 <code>run_code</code> 自测次数	3 次（写完即运行验证）	5 轮合计 2 次（几乎不用）
最终分	~0.446（26/27 题；1 题因合成工具让 worker 死循环卡 2 小时，run 未跑完）	0.536 （1/5 保留——但靠的是加计算器绕开了合成任务）

结论：两个维度分裂——deepseek 会做难的（真合成 + 自测），但合成物选错技术路线（数字 API 覆盖不了叙述性题目）得分反而低；GLM 回避难的但取巧得分更高（0.536 > 0.446）。难度阶梯成功把"真能力"和"取巧刷分"区分开，同时暴露"从零合成好工具"的天花板还很低。→ 下一步动机：FAB 难度阶梯到此走完，按计划把整套机制移植到 GDPval（产文件类任务）。但 GDPval 的评分（LibreOffice 渲染 + 多模态 judge）在本地很重，而 evolve agent 此时还在本地跑——先把它也搬上云，本地才扛得住。

(G) evolve agent 也搬上 E2B 云 VM（此前只有 worker 在云上、evolve agent 在本地跑占内存；commit 542a0ab）

- 单元冒烟测试：71 秒在 VM 内完成"读 worker 副本 → 接回 retrieve_from_filing → 编辑后的目录传回本地 → 预测 JSON 解析"全链路。
- 完整循环验证（FAB 4 题 1 轮）：通过——verdict **EFFECTIVE**（预测修复的题真的修好了，全部实验中首次）且被保留，0.446 → 0.958（注：仅 4 题的易子集，绝对数字勿外推；验证点是链路零错误）。

结论：双 agent 全云化就绪，本地内存近空。→ 下一步动机：基础设施齐了，正式在 GDPval 上做小规模 pilot——验证移植可行性 + 检验预设的"文件产出 gap"（云 VM 模板没装 openpyxl，worker 理论上产不出 .xlsx）是否成立。

(H) GDPval 移植 pilot（2026-07-07 下午，n=8, iters=2, deepseek evolve, 全 E2B）

发现	数据	结论
reference-files 数据修复	同 task 0.264 → 0.663 ；8-task seed = 0.723	fork 原本所有 task <code>reference_files=[]</code> （prompt 承诺附件却没给，worker 编数据）；跑通 <code>fetch_gdpval_reference_files.py</code> 下载 53 个文件（17/30 task）后修复。诊断链自己发现了这个 bug
pip 门控 gap	封 pip（ <code>QEA_E2B_BLOCK_PIP=1</code> ）→ worker sudo 翻盘恢复 pip （产出 xlsx 带完整 openpyxl 签名；绕过已手动复现）	可安装的库 gap 对 sudo+网络 shell worker 不可执行——老教训升级：不止 prompt 削弱、库级 tool-gating 也自愈
无 gap 软 headroom	第 1 轮编辑 0.723 → 0.746（+0.023，低于 0.05 门槛）被回滚；诊断已正确转向内容质量问题	GDPval 对这个 worker 是 capability-sufficient （worker 自身能力已够，没有"被拆掉后可恢复"的空间可供进化）。GDPval 的正确角色 = 迁移评测：把 FAB 上进化好的 worker 冻结后在 GDPval 上测

发现	数据	结论
	(缺 Form-1040 表单生成、表格格式规则)	泛化，而不是直接在 GDPval 上跑进化。FAB 继续当进化主战场

结论： GDPval pilot 三个发现——数据 bug 修复 (+0.4) 让 GDPval 评分从此公平；"可安装的库 gap"这一条路被堵死 (sudo+网络可自愈)；当前 worker + n=8 子集下软 headroom 仅 +0.02。→ **下一步：**不放弃 GDPval——换思路继续找/造 headroom (全量 30 题定位真实弱点集中区、换更弱的 worker 模型让削弱真正 bind、更严格的 VM 门控)，并行扩展其他 benchmark (见 §4 计划 1-2)。GLM 对比 run 依约未启动。

1.4 案例研究 / Case Studies

案例 1 (机制work)： un-firewall 后诊断一句话改变一切。同一个失败 (fab_01: worker 写 "72% U.S. room nights", gold 是 "~75%") ——

- 旧 firewalled critic (禁止提数值) 只能输出 *"lacks structured, precise numerical calculations"*
- 诊断连续 5 轮说"缺计算器" → deepseek 空转 calc.py 6 轮；
- 新 critic (可引用 expected-vs-produced) 直接输出 *"RETRIEVAL gap ... reporting FY2024 data (72% U.S./65% global) instead of the explicitly requested FY2025 data"* → 两个模型 iter1 就接上全部检索工具 → 0.70。

"接近但错的数值 = 检索缺口"这条判别信号，正是 firewall 摧毁掉的东西。值得注意：**信标注释在旧 run 里一直存在但被无视**——绑定 evolve agent 行为的是诊断方向，不是提示。

案例 2 (机制的边界)： T2 里 deepseek 合成了工具、GLM 绕过了任务。deepseek 读到 fetch_page 的 40k 截断，推理后选择写 sec_fact (XBRL company-facts API, 取"精确数字")，run_code 自测 3 次通过——**合成能力成立**。但 company-facts 只覆盖 tagged 数字，覆盖不了叙述性 rubric criteria (如 loyalty 比例在正文里)，所以只 +0.017，还把 fab_24 的 worker 拖到 2h 挂起。GLM 则完全不碰 research.py，加了个 calculator 反而拿到 0.536。**难 tier 把"真能力"和"取巧刷分"分开了。**

2. 分析 / Analysis

1. deepseek ≈ GLM 的"分数"，但行为特征稳定不同。eval 干净时两者都能恢复 headroom 到 ~0.54 (5-iter) / ~0.70 (新 loop iter1)。差异在行为：deepseek **探索多样、敢做难的** (10-iter 里 4 种策略、T2 里真合成工具)；GLM **固着** (10-iter 全 calc.py、T2 拒绝合成) **但擅长取巧** (T2 用 calculator 拿了更高分)。哪个"更好"取决于任务是否强制真能力。
2. **最大的分数杠杆是 harness，不是换模型。** 全程最大提升来自两个 harness 修复：un-firewall + 放开 edit (0.475→0.70 @iter1, 两个模型同样受益)，远大于任何模型间差异 (≤0.06)。换 evolve agent 模型之前，先修诊断信息链。
3. **信息在 diagnosis→evolve 传递中的丢失是首要瓶颈 (AHE 对照证实)。** 精读原 AHE repo：它的 debugger 看真实 test output (expected-vs-got 不 firewall)、evolve agent 每轮可 ship 一批 change (无 L_t=1)、且能直接读全量 detail/raw traces。我们旧设计在这三点上全

都更紧，造成"误导方向 × 每错一步烧一轮"的叠加。对齐后（诊断可引值 + 不限 edit 数），收敛速度 7 轮 → 1 轮。

4. 评测的可信度需要结构性设计，不能靠重试打地鼠。我们经历了三代方案：逐类重试瞬时故障（治标——每次 run 都冒出新的故障类型：先是限流、再是断连、再是 broken pipe）→ **fair-subset keep**（治本——infra 故障的题目直接不参与 keep 比较，故障从此无法冒充退步；run 干净时该机制等于不存在）。附带教训：我们一度用"缓存里没有错误记录"当作"run 干净"的信号，这是**假信号**（瞬时故障根本不写缓存，正确的检查是"27 题是否全部有结果"）；以及同时并行超过 2 个 run 就会触发云端限流、重新引入故障。
5. 从零合成"好"工具的天花板目前很低。run_code 让合成从不可能变可能（T2 deepseek 证明），但合成物（sec_fact）远逊于原实现（passage 检索），甚至有稳定性代价（挂起 worker）。T0/T1（重接现成实现）都能到 0.70，T2 谁都没到 0.55——**5 轮内重建不出同等质量的实现**。

3. 问题与困难（待讨论） / Problems & Open Questions

已解决的困难（详见 commit）：429 风暴与 sandbox 孤儿泄漏（E2B running 上限 ~20，kill 掉的 run 会留孤儿计费）；cache 污染冻结 task 于 0 分（f7b4e62）；transient-masking 翻转 keep 判决（17e8169）；慢 qwen 下顺序 critic ~10min/iter 被误判为挂死（f5d28bd，并行化后 ~90s）。

待讨论的开放项：

1. **keep 判决的语义选择**：我们是同步 hill-climb（每轮 A/B、只升不降），AHE 是 staggered + agent 自主 rollback + best-ever。前者更严格干净，但 plateau 后（incumbent ~0.70）后续小改进过不了相对 noise floor。是否引入 best-ever track 或 staggered 模式？
2. **功能性自测 vs 语法自测**：run_code 目前只验"import+能跑"，验不了"取回的内容对不对"——T2 里 sec_fact 语法全过但覆盖面错了。是否让 evolve agent 拿 1 个真失败 task 的输入做端到端 self-eval？（代价：每 edit 多一次 worker 级调用。）
3. **T2 的公平性**：一个坏的合成工具能把单 worker 拖挂 2h（fab_24），需要 per-worker 迭代/时间上限；且 GLM 的"绕过合成也得分"说明 T2 的验收标准若只看分数，测不出合成能力——是否给"合成了可用检索工具"单独计分？
4. **worker 生成噪声未去**：k=2 只去 judge 噪声；worker 每次重新生成答案的方差仍在（T1 seed 0.534 vs midtier 0.475 部分来自此）。worker k-sampling 代价 ×k，值不值得上？

4. 下周计划 / Next Week's Plan

（原第 1、2 项已在 07-07 完成：evolve-on-E2B 验证通过 §1.3(G)，GDPval 移植 pilot 完成 §1.3(H)。）

1. 继续为 GDPval 设计有 headroom 的 baseline（pilot 的 +0.02 不是终局，按证据依次尝试三条路）：

– (a) 全量 30 题 seed baseline, 定位真实弱点集中区: n=8 pilot 可能低估 headroom——历史数据显示 Accountants/Auditors 职业带存在 rubric mean ~0.10 的"文件生产墙", 且 pilot 里已有 1 题真实 0 分 (Form-1040 表单生成)。若弱点集中在个别职业/题型, 在该子集上进化就有真 headroom。

– (b) 换更弱的 worker 模型: pilot 证明 deepseek worker 能力过强、削弱都被自愈; worker 换弱模型后同样的削弱就会真正 bind, 恢复空间随之出现。

– (c) 更严格的 VM 门控 (备选): no-sudo + 黑洞 PyPI 域名 + 删 pip/ensurepip——堵住"装库自愈", 只留"手写实现"这一条真能力通路。

2. 扩展其他 benchmark (roadmap 既定方向, Benchmark 抽象即插即用): 候选 FinanceBench / FinBen / EconAgent——每个只需接 loader + grader; 多 benchmark 也让"harness 改进是否泛化"可以横向验证。

附录 / Appendix

全部 commits (分支 `worktree-phase5-levelb-mechanism`, 已推送):

`f7b4e62` transient retry+no-cache · `453846e` broken-pipe/5xx 补类 · `130afce` evolve 模型独立切换 · `f5d28bd` 并行 critic+阶段日志 · `17e8169` fair-subset keep · `f245eb6` un-firewall+放开 edit · `6e0a6f2` T1 seed · `9e81249` run_code 自测 · `01657bd` T2 seed · `542a0ab` evolve-on-E2B

结果目录: `results/phase5_fab_*` (旧 run 的 confounded 产物归档于 `_run{1..4}_*` 与 `*_brokenpipe` 等子目录)。

运维备忘: 并发 conc 6、同时最多 2 个 run (3 个会触发 429 masking); E2B kill 掉的 run 要按 template+时间窗清理孤儿 sandbox; 诊断成本 = 每失败 task 一次 judge 调用 (已并行化)。